

Experiment 1: Object Detection and Recognition

Robotics Integration Group Project

College of Computer Science and Technology, Ocean University of China

Personal GitHub: <https://github.com/9161942-boop/robotics-experiment-1>

Abstract—This laboratory experiment builds a complete desktop-object detection system from self-collected data to edge-device deployment. Four classes—mouse, laptop, cup, and phone—were extracted from recorded videos, manually annotated in ISAT, converted to YOLO format, and used to train a YOLOv8n detector from random initialization. No COCO or other external pretrained checkpoint was loaded. The final system displays class names, bounding boxes, confidence scores, object counts, and frame rate on a Jetson Orin, and the ROS2 node publishes the detections on `/detections`. The dataset contains 595 images split into 421 training, 87 validation, and 87 diagnostic-test images. The final validation results are Precision 0.658, Recall 0.748, mAP@0.5 0.766, and mAP@0.5:0.95 0.701. On 80 independent physical-object photographs, four classes were tested with 20 images each; 72 images passed, giving a 90% image-level recognition rate. The Jetson demonstration shows approximately 15 FPS. Data, code, weights, results, video, documentation, and this LaTeX source are archived for acceptance and reproduction.

Index Terms—Object detection, YOLOv8, Jetson, ROS2, independent physical-object test, laboratory report

I. OBJECTIVE AND ACCEPTANCE REQUIREMENTS

The purpose of this experiment is to practice the complete workflow of a practical object detector rather than only calling a pretrained model. The required workflow includes selecting at least two desktop-object classes, collecting and annotating images, training a detection model, running it on Jetson, displaying the class, bounding box and confidence in real time, and publishing the result through ROS2.

The acceptance checks are mapped to explicit evidence. The system must recognize at least two classes simultaneously; the recognition accuracy on at least 20 physical objects must be no lower than 80%; the Jetson display must run at no less than 5 FPS; and the final submission must preserve the dataset, model, programs, result video, running instructions, and typical error cases. This report uses four classes and tests 80 independent photographs, so the physical-object test is four times larger than the minimum requirement.

Acceptance result Four classes are supported; 72 of 80 independent photographs pass (90%); the Jetson demonstration displays approximately 15 FPS; the dataset, model, programs, video, instructions, report, and error cases are archived.

II. PRINCIPLE AND EXPERIMENTAL WORKFLOW

YOLOv8 performs image preprocessing, one network forward pass, and non-maximum suppression (NMS) to produce object classes and bounding boxes. During training, the predicted boxes and class distributions are compared with manually annotated boxes and the network parameters are updated by back-propagation. During validation, an IoU matching rule connects predictions with ground truth and provides Precision, Recall, and mAP. During deployment, the camera frame enters the detector node and the same inference result is sent to the display path and the ROS2 publication path.

The experiment follows the sequence “collect—screen—annotate—convert—train—validate— independent test—deploy—accept—archive.” Original images and ISAT JSON files preserve the annotation basis, `dataset/yolo/` is the training input, the training directory stores parameters and curves, the independent-test directory stores per-image decisions and failures, and the video and documentation directories preserve the edge-device evidence. The workflow therefore connects an input image to a model file, a running program, a result frame, and a final report.

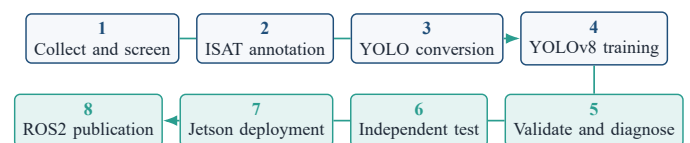


Fig. 1. Complete workflow from source video to an acceptance-ready system

The first four steps determine whether the model receives reliable supervision: screening controls sample quality, ISAT supplies object boundaries, conversion produces normalized YOLO labels, and random-initialization training writes the information into model

parameters. The later steps examine practical usability from different angles. The annotated validation set measures detector quality, the independent photographs test unseen appearances and placements, Jetson checks real-time operation, and ROS2 checks whether another node can consume the result.

III. DATA COLLECTION, ANNOTATION, AND SPLIT

The dataset contains 595 images extracted from desktop-object videos. The four classes are mouse, laptop, cup, and phone; they cover small targets, thin outlines, reflective surfaces, and device appearance variation. Each visible target was manually boxed in ISAT. The ISAT JSON polygons were converted into YOLO detection labels in the form `class_id x_center y_center width height`, with all coordinates normalized to $[0, 1]$.

TABLE I. Dataset split and annotated object counts

Split	Images	Objects	mouse/laptop/cup/phone
Train	421	506	119/85/212/90
Validation	87	104	28/17/42/17
Test diagnostic	87	104	28/17/42/17

The split was made by identifying the source video and assigning contiguous time blocks within each source, rather than randomly scattering adjacent frames. This reduces the chance that nearly identical poses appear in both training and validation. The 80 independent photographs are kept outside the 595-image dataset and contain 20 photographs per class, with different physical objects, orientations, distances, and backgrounds. They therefore provide a separate image-level check of practical recognition.

Annotation quality was checked at three levels. First, each box was inspected to ensure that it covered the visible object without an obvious boundary error. Second, class IDs and image names were checked against `data.yaml`. Third, the converted labels were checked for normalized coordinates, valid class ranges, empty-label anomalies, and image-label correspondence. These checks are particularly useful for reflective phones, thin laptop edges, and small mice, where a slight boundary error can change the training signal.

The collection procedure also kept the data traceable. Frames were extracted from the desktop-object videos, blurred or incomplete frames were removed, and the remaining files retained their source and time order. Before annotation, the four class names were fixed as `mouse`, `laptop`, `cup`, and `phone`; this avoided later remapping of class IDs. After conversion, a small batch of images

was opened together with its label file to confirm that the normalized box was drawn over the intended object. The same check was repeated for small mice, reflective cups, and phones with different orientations because these cases are most sensitive to a one-pixel or one-class labeling error.



Fig. 2. Examples of the four classes with manually annotated bounding boxes

IV. TRAINING CONFIGURATION

The model was trained from `yolov8n.yaml` with random initialization. The training process did not load `yolov8n.pt`, COCO weights, or another external checkpoint. This choice makes the relation between the self-built dataset and the final four-class weights explicit. The main training command was `python train_yolo.py --model yolov8n.yaml --data dataset/yolo/data.yaml --epochs 100 --imgsz 640 --batch 16 --device 0 --workers 0`; the full command record is given below.

Training used 100 epochs, 640-pixel input images, batch size 16, GPU device 0, zero data-loader workers, seed 42, automatic optimizer selection, standard YOLO augmentation, and early-stopping patience of 25. Only the training and validation splits were used for parameter optimization. The diagnostic test split was retained until training finished, so it was not used to choose the checkpoint.

The training directory preserves the argument file, loss and metric curves, confusion matrices, and the final weight. The curves are used to inspect convergence and late-stage stability, while the confusion matrix shows whether a class is systematically confused with another class. Keeping both forms of evidence is more informative than reporting only one final scalar.

A. Command-level operation record

The following commands record what was executed at each important stage and what each command produced:

- 1) **Convert annotations.** Run `python convert_isat_to_yolo.py --images-root dataset/images --output-root dataset/yolo --classes mouse,laptop,cup,phone`. The script reads ISAT JSON files, recomputes boxes from polygons, performs coordinate checks, assigns the temporal train/validation/test split, and writes `data.yaml`, image files, label files, a manifest, and `conversion_summary.json`.
- 2) **Train from scratch.** Run `python train_yolo.py --model yolov8n.yaml --data dataset/yolo/data.yaml --epochs 100 --imgsz 640 --batch 16 --device 0 --workers 0`. The YAML model definition creates a randomly initialized network; the run directory records the training configuration, curves, confusion matrix, and `weights/best.pt`.
- 3) **Diagnose annotated images.** Run `python evaluate_yolo.py --weights models/best.pt --images dataset/yolo/images/test --labels dataset/yolo/labels/test --output results/evaluation_final --confidence 0.25 --match-iou 0.50 --device 0`. This produces `summary.json`, `per_image.csv`, and visualized error cases without changing the weights.
- 4) **Run real-time detection.** On Jetson, run `python realtime_detect.py --weights models/best.pt --source 0 --imgsz 320 --confidence 0.35 --device 0 --output results/videos/demo.mp4`. The program opens the USB camera, performs inference, draws boxes and confidence scores, calculates a sliding FPS, and saves an annotated video when requested.
- 5) **Publish through ROS2.** Start `python ros2_detector_node.py --weights models/best.pt --image-topic /camera/image_raw --output-topic /detections --device 0`, then inspect one message with `ros2 topic echo /detections --once`. The node converts `sensor_msgs/Image` using `cv_bridge` and publishes `vision_msgs/Detection2DArray`.

These commands separate data generation, parameter optimization, offline diagnosis, edge-device inference, and inter-node communication. That separation is useful during acceptance because a failure in one stage can be located from its output directory instead of being confused with a model-quality problem.

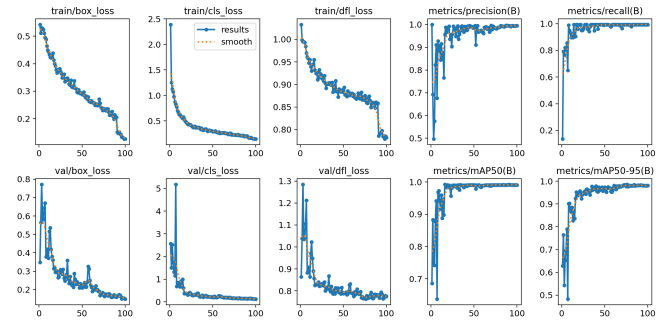


Fig. 3. Training losses and evaluation metrics over 100 epochs

V. VALIDATION AND INDEPENDENT PHYSICAL-OBJECT RESULTS

The final four-class model produced the following validation results on `dataset/yolo/`: Precision 0.658, Recall 0.748, `mAP@0.5` 0.766, and `mAP@0.5:0.95` 0.701. At the class level, `mAP@0.5:0.95` was 0.723 for mouse, 0.877 for laptop, 0.428 for cup, and 0.776 for phone. The lower cup value is consistent with its reflective rim, handle, and similarity to the desktop background.

TABLE II. Final validation metrics

Dataset	P	R	mAP@0.5	mAP@0.5:0.95
Validation	0.658	0.748	0.766	0.701

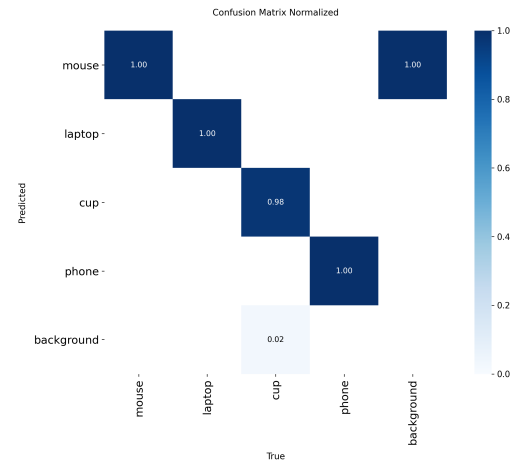


Fig. 4. Normalized confusion matrix for the four target classes

The independent test contains 80 photographs, 20 per class. A photograph passes when the predicted class is correct and at least one detection box covers the target body. A miss, a wrong class, or a background-only detection is counted as a failure. This is an image-level physical-object acceptance rule; it is intentionally kept separate from IoU-based mAP.

For the annotated diagnostic set, `evaluate_yolo.py` used a confidence threshold of 0.25 and a matching IoU of 0.50. It saved one row per image in `per_image.csv`, including the predicted class, confidence, match status, and error category. The independent-photo script then applied the simpler pass rule above and copied failed files into `results/independent_test_80/errors/`. Thus, the two evaluations answer different questions: the first checks box quality against known annotations, while the second checks whether an unseen physical object is recognized well enough for demonstration.

TABLE III. Independent physical-object test

Class	Tested	Passed	Rate
cup	20	17	85%
laptop	20	19	95%
mouse	20	18	90%
phone	20	18	90%
Total	80	72	90%

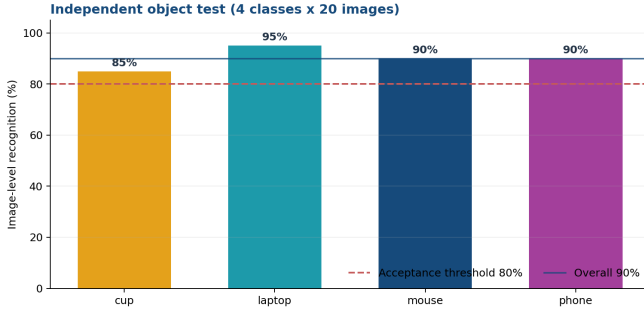


Fig. 5. Class-wise pass rates for the 80 independent photographs and the 80% threshold

All four classes exceed the 80% class-level threshold. Laptop reaches 95% because its large area and clear outline are easy to localize. Cup is the weakest class at 85%, while mouse and phone both reach 90%. The result of 72/80 is therefore suitable for the course physical-object check, while the validation metrics above remain the appropriate evidence for annotated detector quality.

VI. JETSON DEPLOYMENT AND ROS2 PUBLICATION

The final four-class weight was copied to Jetson Orin and loaded by `realtime_detect.py`. The program reads a USB camera, performs GPU inference, draws bounding boxes, class names and confidence scores, counts objects by class, computes a sliding-average FPS, and can save an annotated MP4. The camera and output video use 640×480, while inference uses 320 pixels to keep the complete read—infer—NMS—draw—display chain responsive.

The Jetson demonstration video displays approximately 15 FPS, which is above the 5 FPS acceptance threshold. The confidence threshold was set to 0.35 for the demonstration so that very low-confidence boxes were less distracting. This display threshold changes the visual filtering only; it does not alter the trained weights or the offline validation metrics.

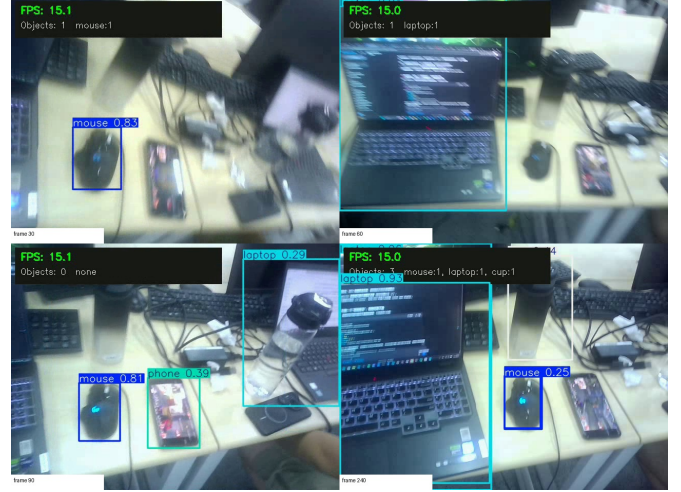


Fig. 6. Frames from the Jetson demonstration video, showing boxes, classes, confidence, object counts, and FPS

The ROS2 node `ros2_detector_node.py` subscribes to `/camera/image_raw` as `sensor_msgs/Image`, converts frames with `cv_bridge`, runs the detector, and publishes `vision_msgs/Detection2DArray` on `/detections`. Each detection contains the class name, confidence, and pixel-coordinate bounding-box information. The documented verification command is `ros2 topic echo /detections --once`; the source and startup instructions are included in the submission package so the interface can be checked when the Jetson environment is available.

The deployment check followed a fixed sequence. First, the selected `best.pt` was copied to the Jetson project directory and the Python dependencies were verified. Second, the USB camera was opened and a short run was recorded with the same 320-pixel inference setting used for the demonstration. Third, the overlay was checked for class names, confidence values, boxes, object counts, and the FPS counter. Finally, the ROS2 node was started with the camera topic and the output topic shown above. This sequence separates a missing camera or topic from a detector error and leaves a video artifact for later inspection.

VII. TYPICAL ERROR CASES AND DISCUSSION

Eight of the 80 independent photographs fail: three cups, one laptop, two mice, and two phones. The cup

failures are mainly related to low contrast between the object edge and the desktop, reflective surfaces, and handle or rim appearance. The laptop failure is associated with an oblique view and screen reflection. The mouse failures involve small apparent size or partial occlusion. The phone failures involve new-device appearance, low contrast, and changes in placement direction.



Fig. 7. Independent-test failure example: the true class is cup, but a background laptop is predicted as the target (04_cup.jpg)

The example illustrates why image-level failure analysis matters. The detector did produce a high-confidence box, but the box covered a background laptop rather than the cup. We therefore record errors along three dimensions: class, pose, and background. This makes the next data-collection step concrete: add difficult cup viewpoints and reflective backgrounds, add occluded or small mouse examples, and add phone orientations and newer device appearances. The current errors do not indicate that every class needs the same amount of additional data.

VIII. SUBMISSION MATERIALS AND REPRODUCTION

The material package is organized as “input—model—program—result—documentation—report.” The dataset contains the YOLO images and labels plus the 80 independent photographs. The model directory contains the final four-class `best.pt`. The program directory contains training, evaluation, real-time detection, ROS2, and Jetson deployment scripts. The result directories contain training curves, confusion matrices, per-image independent-test records, failures, and the Jetson videos. The report directory contains this PDF and its LaTeX source.

The main reproduction steps are:

- 1) Install the Windows training dependencies from `requirements.txt` and verify the YOLO dataset configuration.
- 2) Run `convert_isat_to_yolo.py` when the ISAT annotations need to be regenerated, then run the training command in Section IV.
- 3) Run `evaluate_yolo.py --device 0` for the annotated validation and diagnostic outputs.
- 4) Copy `models/best.pt` to Jetson, start `realtime_detect.py`, and check the overlay and displayed FPS.
- 5) Start `ros2_detector_node.py` and use `ros2 topic echo /detections --once` to inspect the published message.

The report, dataset, model, code, video, instructions, and error cases are stored in the public repository <https://github.com/9161942-boop/robotics-experiment-1>. A SHA-256 manifest is included in the complete package. The report records the final four-class model and the independent-test statistics; older exploratory results are not used as acceptance evidence.

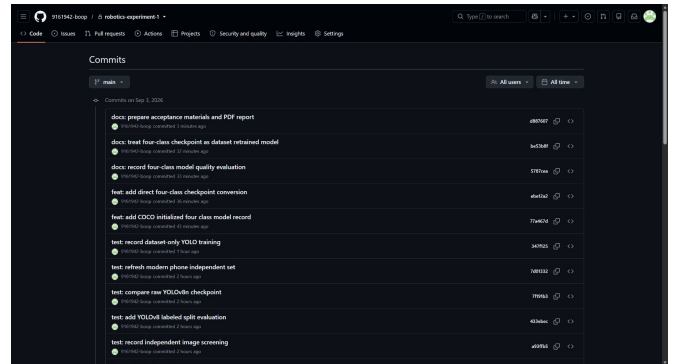


Fig. 8. GitHub main-branch commit history used to preserve the experiment process

IX. ACCEPTANCE SUMMARY

TABLE IV. Acceptance requirements and corresponding evidence

Requirement	Evidence	Status
At least two classes	Four-class model and simultaneous detections in the Jetson video	Met
20 objects, at least 80%	Independent test: 72/80, 90%, with class-wise statistics	Met
At least 5 FPS on Jetson	Jetson video overlay shows approximately 15 FPS	Met
Save results and typical errors	Training plots, per-image records, failure images, and video	Met
ROS2 publication	<code>/detections</code> publisher and startup instructions	Implemented

A. Evidence traceability

Each acceptance statement is supported by a concrete file rather than by a screenshot alone. The dataset split can be checked in `dataset/yolo/data.yaml` and `conversion_summary.json`; the training choice and hyperparameters are recorded in the run configuration; the independent accuracy is summarized in `results/independent_test_80/summary.json`; and the eight failed photographs are retained in the corresponding `errors/` directory. The video, program, and ROS2 documentation then connect these offline records to the edge-device demonstration.

TABLE V. Operation, program, and saved evidence

Stage	Main program or command	Saved evidence
Conversion	<code>convert_isat_to_yolo.py</code>	YOLO images, labels, <code>data.yaml</code> , manifest
Training	<code>train_yolo.py</code>	Arguments, curves, confusion matrix, <code>best.pt</code>
Diagnosis	<code>evaluate_yolo.py</code>	Summary JSON, per-image CSV, error images
Deployment	<code>realtime_detect.py</code>	Annotated video and displayed FPS
Interface	<code>ros2_detector_node.py</code>	<code>/detections</code> topic instructions

SUMMARY AND REFLECTIONS

This experiment completed a traceable path from self-collected images to an edge-device detection system. The main summary and reflections are:

- **Data and training:** The 595-image dataset was manually annotated and divided into 421 training, 87 validation, and 87 diagnostic-test images. Starting from `yolov8n.yaml` rather than an external checkpoint made the training source explicit and kept the final four-class weight connected to the self-built dataset.
- **Recognition quality:** The validation metrics quantify the quality of predicted boxes, while the 80 independent photographs evaluate practical recognition. The 72/80 result (90%) shows that the system meets the course threshold on unseen physical objects, and the class-wise table reveals where further sampling would be most useful.

- **Deployment and interface:** The Jetson video demonstrates the complete camera-to-display chain at approximately 15 FPS. The ROS2 node packages the same detections as `Detection2DArray` on `/detections`, so the result can be consumed by later tracking, counting, or manipulation modules.

- **Lessons and improvements:** The remaining failures are concentrated in reflective cups, small or occluded mice, and phones with changed appearance or orientation. Future work should add these difficult conditions and evaluate TensorRT optimization on Jetson, while keeping the independent test set fixed for comparison.

The practical value of this experiment is the complete engineering chain: data are traceable to labels, labels are traceable to weights, weights are traceable to a running program, and the program is traceable to saved videos and ROS2 messages. This organization makes the system easier to demonstrate, reproduce, and improve than a detector accompanied only by a single accuracy number.

Practical reflection

The experiment also clarified why different measurements must be kept separate. The validation set uses manually annotated boxes and IoU matching, so its Precision, Recall, and mAP describe localization and classification quality under a fixed protocol. The independent photographs use a simple image-level pass rule, so their 90% result answers a different question: whether a new physical object is recognized well enough for the course demonstration. Reporting both values prevents a high score on correlated video frames from being mistaken for real-world object accuracy.

From an implementation perspective, the most useful practice was recording the output of every stage. A conversion error appears in the label or manifest checks, a training error appears in the run directory, a visual failure appears in the per-image CSV and error folder, and a deployment problem appears in the video or ROS2 topic. This division makes debugging more direct and gives the next experiment a clear starting point. The final system is therefore both an accepted prototype and a reproducible baseline for adding difficult samples and optimizing Jetson inference.